CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY (CHARUSAT)

Chandubhai S. Patel Institute of Technology (CSPIT)

Department of Computer Science & Engineering

ITUE203 – Web Development Frameworks

PRACTICAL 12

Event Management with Secure Poster Upload and Event Registration

PHP 8 · MySQLi prepared statements · PHP Fileinfo · MariaDB (studenthub_db) · Bootstrap 5

Corrected submission – implemented in the integrated StudentHub portal (studenthub/)

Student Name	Shreyas
Roll Number	25CS023
Student ID	25CS023
Email	25cs023@charusat.edu.in
Course	ITUE203 – Web Development Frameworks
Programme	B.Tech Computer Science & Engineering
Semester / Academic Year	3 / 2026-27
Date of Submission	
Faculty Signature	

Corrected report. This report replaces the original Practical 12 report (`Practical-12/PRAC_12_ITUE203_25CS023.docx`, kept unchanged in the repository). The original report describes a standalone application in `Practical-12/` with its own database `studenthub_prac12`, its own `admins` table and session. This corrected version describes event management as it is now implemented in the integrated portal: the Campus Events tab of the Admin Console works on the portal's own `events` table (extended by migration 003), uses the portal login, and students register for events on the Campus Events page. All screenshots in this report were taken from the canonical integrated application `studenthub/` with Microsoft Edge on disposable test databases with fictional records; all test results are copied from the saved test outputs of the repository.

1. Aim and Objectives

To build the event module of StudentHub in which administrators publish campus events with a poster image, safely, inside the integrated portal – and in which students register for these events without overbooking.

- Add, view, update and delete events (title, category, date, optional start time, location, optional capacity, Open/Closed status, optional description, optional poster) in the Admin Console.
- Upload event posters and validate the file type (JPEG, PNG, WebP) and file size (max 2 MB) on the server – from the file content, not from the name or the browser.
- Show clear success and failure messages for every action.
- Restrict event management to administrators (permission `events.manage`) through the portal's single login.
- Intermediate extension: event categories from a database table, an Open/Closed filter, an upcoming/past filter and a search.
- Let students with a linked student record register for an event and cancel the registration (Practicals 8 and 12); the capacity is never exceeded, also for simultaneous requests.
- Use MySQLi prepared statements, output escaping and a CSRF token on every POST throughout.

2. Problem Definition

Uploading files is one of the most dangerous features of a web application: if the server trusts the file name, the extension or the browser's `accept` attribute, an attacker can upload a PHP script and run it, an SVG file with JavaScript, a disguised executable or a huge file. A carelessly built edit or delete function can also destroy the wrong file, follow a path such as `../db.php`, or lose the old poster when an update fails. The module must therefore validate every poster on the server, store it safely, change files only after the database change succeeded, and let only authorised administrators manage events.

The integrated portal adds three constraints. (1) The portal already has an `events` table that the Campus Events page shows; some existing events store an older poster path (for example `uploads/odoo_hackathon.jpg`) whose file does not exist. (2) The `registrations` table (Practical 8 foundation) references `events` with `ON DELETE CASCADE`, so deleting an event would silently remove its registration history. (3) Registrations must respect the capacity even when several students click *Register* at the same moment. The standalone design of the original report (own database and own administrator login) could not be copied; its rules had to be brought into the existing tables and login.

3. Tools and Technologies

Tool	Use in this practical
PHP 8.2.12 (XAMPP)	Admin Console, Campus Events page, upload handling, `poster.php`
MySQLi (exception mode)	Every query with values as a prepared statement; transactions with row

	locks
PHP Fileinfo (`finfo`) + `getimagesizefromstring()`	Real MIME type and image type detected from the file content
MariaDB 10.4.32 (XAMPP)	Shared database `studenthub_db`; migration 003 (event categories, status, unique poster)
Bootstrap 5.3.2 + Bootstrap Icons	Admin Console layout (CDN); the Campus Events page uses the portal's `global.css`
Python 3	`tests/test_events.py` (HTTP and multipart upload tests) and `database/tests/test_003_event_management.py`
Microsoft Edge (headless) + Playwright	`tests/browser_events.py` – browser checks and the screenshots
Git / GitHub	Commit `ba11795`, pushed to the project repository

4. Place in the Integrated Portal

Event management is the **Campus Events** tab of the Admin Console; event registration is part of the students' **Campus Events** page (`notifications.php`, menu entry *Notices*). Both use the same include files.

File / route	Role in Practical 12
`admin_dashboard.php?tab=events`	Event list with search and filters; `view=add_event`, `view=edit_event&id=N`, `view=delete_event&id=N`, `view=event_registrations&id=N` (GET only shows them)
`admin_dashboard.php` (POST, multipart)	`action=add_event`, `update_event`, `delete_event` – CSRF token and permission `events.manage` required
`includes/event_management.php`	`event_filters()`, `validate_event()`, `save_event()`, `delete_event()`, `validate_poster()`, `poster_structure_ok()`, `store_poster()`, `poster_path()`, `delete_poster_file()`, `discard_new_poster()`
`includes/event_registration.php`	`register_student_for_event()` (transaction), `cancel_event_registration()`, `linked_student()`
`notifications.php`	Campus Events page: upcoming events with poster, seats and Register / Cancel buttons; past events
`poster.php?event=N`	The only way to view a poster; signed-in users only
`uploads/` + `uploads/.htaccess`	Poster folder (`STUDENTHUB_UPLOAD_DIR` overrides it); denied over HTTP and PHP disabled on Apache
`database/migrations/003_event_management.sql`	`event_categories` + foreign key, `events.status`, `events.updated_at`, UNIQUE `events.poster` (with rollback)
`audit_logger.php`	CREATE_EVENT, UPDATE_EVENT, DELETE_EVENT, UPLOAD_REJECTED, EVENT_REGISTER, EVENT_REGISTRATION_CANCEL

Permissions (`includes/auth.php`): `events.manage` belongs to the role `admin` only; `portal.view` (Campus Events page and posters) to admin, editor and student. Only a `student` account with a linked student record (`students.user_id`) can register – administrators cannot register as students. Students and editors get the common HTTP 403 page for the Campus Events tab and every event POST, logged as ACCESS_DENIED. The tab shares the portal's session, CSRF token and audit log with all other pages.

5. Database Design and Relationships

Practical 12 extends the portal's existing `events` table instead of creating a new one. After migration 003 its structure (from `database/studenthub_demo.sql`) is:

Column	Type	Notes
id	INT(11) AUTO_INCREMENT	Primary key
title	VARCHAR(150) NOT NULL	3–150 characters (application)
event_date / event_time	DATE NOT NULL / TIME NULL	Index `idx_events_date`; the start time is optional
location	VARCHAR(150) NOT NULL	2–150 characters (application)
capacity	SMALLINT UNSIGNED NULL	CHECK `chk_events_capacity`: NULL (no limit) or > 0
status	ENUM('open','closed') NOT NULL DEFAULT 'open'	Migration 003; closed = no new registrations; index
description	TEXT NULL	Optional, at most 2000 characters (application)
category	VARCHAR(50) NOT NULL	FK `fk_events_category` → `event_categories.name` (ON UPDATE CASCADE, ON DELETE RESTRICT)
poster	VARCHAR(255) NULL	UNIQUE `uq_events_poster` (migration 003): two events can never share a poster file
created_at / updated_at	TIMESTAMP	`updated_at` set automatically on every change (migration 003)

Table / relationship	Rule	Effect
event_categories 1 : n events	`fk_events_category`, ON DELETE RESTRICT	Category drop-downs come from the table; a used category cannot be deleted
events 1 : n registrations	`fk_registrations_event` ON DELETE CASCADE	The portal refuses to delete events with registrations, so history is never lost through the interface
students 1 : n registrations	`fk_registrations_student` ON DELETE CASCADE; UNIQUE (student_id, event_id)	One registration per student and event; status registered / attended / cancelled
users 1 : 0..1 students	`students.user_id`	Decides which student record a signed-in student registers with

5.1 Migration 003 – event management

The migration first checks its preconditions (migration 002 recorded, the expected column types, no two events with the same poster value, no empty category) and changes nothing if one fails. It creates `event_categories` with the six categories of the original Practical 12 **plus every category already used** (the live database keeps *Conference*), adds `status`, `updated_at`, the unique poster key, two indexes and the category foreign key, verifies them and records version `003`. No event value is changed; it is idempotent and `003_event_management.down.sql` removes exactly these changes. According to `database/README.md` it was applied to the live database on 31 Oct 2026 after a verified backup; the read-only live check lists the categories Conference, Cultural, Hackathon, Seminar, Sports, Technical and Workshop.

database/migrations/003_event_management.sql (lines 54–81) – categories, new columns, unique poster and foreign key

```
54  CREATE TABLE IF NOT EXISTS event_categories (
55      name        VARCHAR(50)  NOT NULL,
56      description VARCHAR(200) DEFAULT NULL,
57      PRIMARY KEY (name)
58  ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
59
60  INSERT IGNORE INTO event_categories (name, description) VALUES
```

```
61      ('Cultural',  'Music, dance, drama and festivals'),
62      ('Hackathon', 'Team coding competitions'),
63      ('Seminar',   'Talks and guest lectures'),
64      ('Sports',    'Matches and tournaments'),
65      ('Technical', 'Technical talks and project expos'),
66      ('Workshop',  'Hands-on training sessions');
67  INSERT IGNORE INTO event_categories (name) SELECT DISTINCT category FROM events;
68
69  -- ----------------------------------------------------------------------------------------------------
    -----------------
70  -- 2. Events: status, updated_at, unique poster, category foreign key
71  -- ----------------------------------------------------------------------------------------------------
    -----------------
72  ALTER TABLE events
73      ADD COLUMN IF NOT EXISTS status ENUM('open', 'closed') NOT NULL DEFAULT 'open' AFTER capacity,
74      ADD COLUMN IF NOT EXISTS updated_at TIMESTAMP NULL DEFAULT NULL ON UPDATE CURRENT_TIMESTAMP AFTER
    created_at,
75      ADD UNIQUE KEY IF NOT EXISTS uq_events_poster (poster),
76      ADD KEY IF NOT EXISTS idx_events_status (status),
77      ADD KEY IF NOT EXISTS idx_events_category (category);
78
79  ALTER TABLE events
80      ADD CONSTRAINT fk_events_category FOREIGN KEY IF NOT EXISTS (category) REFERENCES event_categories
    (name)
81          ON UPDATE CASCADE ON DELETE RESTRICT;
```

6. Implementation

6.1 Event list, search and filters

The Campus Events tab lists the events 10 per page with a poster thumbnail (or a generic image icon), title, category, date and time, location, seats (*taken / capacity* or *no limit*), the Open/Closed badge and the actions Edit, Registrations (with the number of registrations) and Delete. `event_filters()` accepts a category only from `event_categories`, a status only `open`/`closed` and a date filter only `upcoming`/`past`; `event_filter_sql()` builds a WHERE clause with `?` parameter markers; the search text is matched against title, location and description with `%` and _ escaped. The same functions are later reused by the Practical 13 JSON API.

6.2 Add and edit with server-side validation

`validate_event()` checks every field on the server: title 3–150 characters, a category from the table, a real date in `YYYY-MM-DD` between 2000 and 2100, an optional time `HH:MM`, location 2–150 characters, capacity a whole number 1–65535 or empty for no limit, status Open or Closed and a description of at most 2000 characters. Invalid input is answered with HTTP 422, a message under each field and the typed values (HTML-escaped). When an event is edited, the capacity can never be set below the seats already taken – `save_event()` re-checks this while the event row is locked, so a registration arriving at the same moment cannot slip through.

studenthub/includes/event_management.php (lines 251–276) – update inside a transaction with the capacity re-check

```
251             // lock the event, then re-check the capacity against the seats taken at this moment
252             $stmt = $db->prepare('SELECT poster FROM events WHERE id = ? FOR UPDATE');
253             $stmt->bind_param('i', $id);
254             $stmt->execute();
255             $row = $stmt->get_result()->fetch_assoc();
256             if ($row === null) {
257                 $db->rollback();
258                 return ['errors' => ['title' => 'This event no longer exists.']];
259             }
260             if ($clean['capacity'] !== null) {
261                 $stmt = $db->prepare("SELECT COUNT(*) FROM registrations WHERE event_id = ? AND
    status <> 'cancelled' LOCK IN SHARE MODE");
262                 $stmt->bind_param('i', $id);
263                 $stmt->execute();
264                 $taken = (int) $stmt->get_result()->fetch_row()[0];
265                 if ($clean['capacity'] < $taken) {
```

```
266                     $db->rollback();
267                     return ['errors' => ['capacity' => "The capacity cannot be lower than the $taken
    seats already taken."]];
268                 }
269             }
270             $new_poster = $poster ?? $row['poster'];
271             $stmt = $db->prepare('UPDATE events SET title = ?, category = ?, event_date = ?,
    event_time = ?, location = ?, capacity = ?,
272                                   status = ?, description = ?, poster = ? WHERE id = ?');
273             $stmt->bind_param('sssssisssi', $clean['title'], $clean['category'],
    $clean['event_date'], $clean['event_time'], $clean['location'],
274                               $clean['capacity'], $clean['status'], $clean['description'],
    $new_poster, $id);
275             $stmt->execute();
276             $old_poster = $poster !== null ? $row['poster'] : null;
```

6.3 Poster upload and validation

All checks run on the server in `validate_poster()`; the `accept` attribute of the file field is only a convenience. The checks are applied in this order, and the first failure ends the check with a clear message:

Step	Check	Rejects for example
1	PHP upload error code	file above the server limit ("too large"), partial upload, missing temp folder
2	`is_uploaded_file()`	a path that was not uploaded in this request
3	Size: not empty, at most 2 MB (POSTER_MAX_BYTES)	an empty file; a 2.3 MB PNG
4	Extension `.jpg` / `.jpeg` / `.png` / `.webp` (any letter case)	GIF, SVG with JavaScript, a PHP script
5	The whole file can be read	a file blocked by security software ("could not be read as an image")
6	Fileinfo MIME type of the content is JPEG, PNG or WebP	a PHP script renamed to `.jpg`, `shell.php.jpg`, a text file sent with a forged `image/png` type
7	MIME type matches the extension	PNG content named `.jpg` ("does not match its content")
8	`getimagesizefromstring()` + structure check `poster_structure_ok()` (PNG chunks with CRC up to IEND, JPEG start/end markers, WebP RIFF length)	a truncated PNG ("not a valid image or is damaged")
9	Width and height 1–6000 px	gigantic dimensions
10	No `<?php` or `<?=` inside the file	a valid JPEG with embedded PHP code

studenthub/includes/event_management.php (lines 359–396) – content-based checks of validate_poster()

```
359     $tmp = (string) ($file['tmp_name'] ?? '');
360     if ($tmp === '' || !is_uploaded_file($tmp)) {
361         return ['ok' => false, 'none' => false, 'error' => 'The poster upload was not valid.'];
362     }
363     $size = filesize($tmp);
364     if ($size === false || $size === 0) {
365         return ['ok' => false, 'none' => false, 'error' => 'The poster file is empty.'];
366     }
367     if ($size > POSTER_MAX_BYTES) {
368         return ['ok' => false, 'none' => false, 'error' => sprintf('The poster is %.1f MB. The
    maximum size is 2 MB.', $size / 1048576)];
369     }
370     $extension = strtolower(pathinfo((string) ($file['name'] ?? ''), PATHINFO_EXTENSION));
371     if (!isset(POSTER_EXTENSIONS[$extension])) {
372         return ['ok' => false, 'none' => false, 'error' => 'Invalid file type. Only JPEG, PNG and
    WebP images (.jpg, .jpeg, .png, .webp) are allowed.'];
373     }
374     $content = @file_get_contents($tmp);
375     if ($content === false || strlen($content) !== $size) {
376         return ['ok' => false, 'none' => false, 'error' => 'Invalid file type. The file could not be
    read as an image.'];
```

```
    }
    $mime = (new finfo(FILEINFO_MIME_TYPE))->buffer($content);
    if (!in_array($mime, POSTER_EXTENSIONS, true)) {
        return ['ok' => false, 'none' => false, 'error' => 'Invalid file type. The file content is not a JPEG, PNG or WebP image.'];
    }
    if ($mime !== POSTER_EXTENSIONS[$extension]) {
        return ['ok' => false, 'none' => false, 'error' => 'The file extension does not match its content (.' . $extension . ' file containing ' . $mime . ').'];
    }
    $info = @getimagesizefromstring($content);
    if ($info === false || !isset(POSTER_IMAGE_TYPES[$info[2]]) || image_type_to_mime_type($info[2]) !== $mime
        || !poster_structure_ok($content, POSTER_IMAGE_TYPES[$info[2]])) {
        return ['ok' => false, 'none' => false, 'error' => 'The poster is not a valid image or is damaged.'];
    }
    if ($info[0] < 1 || $info[1] < 1 || $info[0] > POSTER_MAX_PIXELS || $info[1] > POSTER_MAX_PIXELS) {
        return ['ok' => false, 'none' => false, 'error' => 'The poster must be between 1 and ' . POSTER_MAX_PIXELS . ' pixels wide and high.'];
    }
    if (stripos($content, '<?php') !== false || stripos($content, '<?=') !== false) {
        return ['ok' => false, 'none' => false, 'error' => 'The poster was rejected because it contains embedded code.'];
    }
    return ['ok' => true, 'tmp' => $tmp, 'ext' => POSTER_IMAGE_TYPES[$info[2]]];
```

6.4 Safe storage, replacement and delivery

The original file name is never used. `store_poster()` names the file with 32 random hexadecimal characters (`random_bytes(16)`) plus the extension of the *detected* type and moves it with `move_uploaded_file()`; only this name is stored in `events.poster`. `poster_path()` accepts only names matching `^[a-f0-9]{32}\.(jpg|png|webp)$` that really lie inside the upload folder, so older values such as `uploads/odoo_hackathon.jpg` or a value like `../db.php` can never reach another file – such events simply show a generic image icon.

studenthub/includes/event_management.php (lines 433–460) – random names and the path check

```
function store_poster(array $valid): ?string
{
    $dir = poster_dir();
    if (!is_dir($dir) && !mkdir($dir, 0755, true)) {
        error_log('StudentHub: poster folder missing and could not be created');
        return null;
    }
    do {
        $name = bin2hex(random_bytes(16)) . '.' . $valid['ext'];
    } while (file_exists($dir . DIRECTORY_SEPARATOR . $name));
    if (!move_uploaded_file($valid['tmp'], $dir . DIRECTORY_SEPARATOR . $name)) {
        error_log('StudentHub: move_uploaded_file failed for a poster');
        return null;
    }
    @chmod($dir . DIRECTORY_SEPARATOR . $name, 0644);
    return $name;
}

// Full path of a stored poster, or null if the name is not exactly "<32 hex>.<jpg|png|webp>" or the file is missing.
// Older values such as "uploads/odoo_hackathon.jpg" or "../db.php" never pass, so no other file can be reached.
function poster_path(string $name): ?string
{
    if (!preg_match(POSTER_NAME_PATTERN, $name)) {
        return null;
    }
    $dir = realpath(poster_dir());
    $path = $dir === false ? false : realpath($dir . DIRECTORY_SEPARATOR . $name);
    return ($path !== false && dirname($path) === $dir && is_file($path)) ? $path : null;
```

The console's POST handler keeps files and database consistent: the poster is validated together with the fields, stored only when everything is valid, then the event is saved; if saving fails the new file is removed again (`discard_new_poster()`), and only after a successful update is the old poster deleted (`delete_poster_file()`, which also checks that no event still uses it). A rejected poster is logged as UPLOAD_REJECTED.

studenthub/admin_dashboard.php (lines 112–146) – POST handler of the Campus Events tab

```
112      // Campus events (Practical 12): add, edit and delete with a validated poster upload – admins
     only (events.manage)
113      if (in_array($action, ["add_event", "update_event", "delete_event"], true)) {
114          require_permission("events.manage");
115          $current_tab  = "events";
116          $target_id    = event_id_from($_POST["id"] ?? "");
117          $event_record = $target_id > 0 ? find_event($conn, $target_id) : null;
118          if ($action !== "add_event" && $event_record === null) {
119              flash("error", "That event was not found."); redirect("admin_dashboard.php?tab=events");
120          }
121          if ($action === "delete_event") {
122              $deleted = delete_event($conn, $target_id);
123              if ($deleted["ok"]) {
124                  delete_poster_file($conn, $deleted["poster"]);
125                  log_audit($conn, $current_user["id"], $current_user["username"], "DELETE_EVENT",
     "Deleted event ID $target_id ({$event_record["title"]})", "SUCCESS");
126                  flash("success", "Event “{$event_record["title"]}” was deleted.");
127              } else {
128                  flash("error", $deleted["error"]);
129              }
130              redirect("admin_dashboard.php?tab=events");
131          }
132          $existing = $action === "update_event" ? $event_record : null;
133          [$clean, $errors] = validate_event($_POST, $event_categories, $existing);
134          $poster = validate_poster($_FILES["poster"] ?? null);
135          if (!$poster["ok"] && !$poster["none"]) {
136              $errors["poster"] = $poster["error"];
137              log_audit($conn, $current_user["id"], $current_user["username"], "UPLOAD_REJECTED",
138                  "Poster rejected for " . ($existing ? "event ID $target_id" : "a new event") . ": " .
     $poster["error"], "FAILED");
139          }
140          $stored = null;
141          if (!$errors && $poster["ok"] && ($stored = store_poster($poster)) === null) {
142              $errors["poster"] = "The poster could not be saved. Please try again.";
143          }
144          $saved = $errors ? ["errors" => $errors] : save_event($conn, $clean, $existing ? $target_id :
     null, $stored);
145          if (isset($saved["errors"])) {
146              discard_new_poster($stored);
```

The poster folder is never served directly. On Apache, `uploads/.htaccess` denies every request (`Require all denied`), switches the PHP engine off, removes the PHP handlers and types and disables CGI and directory listings. Posters are delivered only by `poster.php`, which takes an **event ID** – never a file name – and requires a signed-in user.

studenthub/poster.php (lines 10–26) – poster delivery with nosniff and a sandbox CSP

```
10  require_permission("portal.view");
11  $event = ($id = event_id_from($_GET["event"] ?? "")) > 0 ? find_event($conn, $id) : null;
12  $path  = $event !== null && $event["poster"] !== null ? poster_path((string) $event["poster"]) :
    null;
13  if ($path === null) {
14      http_response_code(404);
15      header("Content-Type: text/plain; charset=utf-8");
16      echo "Poster not found.\n";
17      exit;
18  }
19  $types = ["jpg" => "image/jpeg", "png" => "image/png", "webp" => "image/webp"];
20  $ext = pathinfo($path, PATHINFO_EXTENSION);
21  header("Content-Type: " . $types[$ext]);
22  header("Content-Length: " . filesize($path));
23  header('Content-Disposition: inline; filename="poster.' . $ext . '"');
24  header("Content-Security-Policy: default-src 'none'; sandbox");
25  header("Cache-Control: private, max-age=300");
```

```
 26  readfile($path);
```

6.5 Deleting events

The Delete action first opens a confirmation panel. For an event **with** registrations the panel explains that it cannot be deleted and offers *Edit the event* (to close it) instead of a delete button; a forged POST is refused by `delete_event()` as well, which counts the registrations while the event row is locked. An event without registrations is deleted after confirmation, and its poster file is removed.

studenthub/includes/event_management.php (lines 291–315) – delete only without registrations

```
291  function delete_event(mysqli $db, int $id): array
292  {
293      $db->begin_transaction();
294      try {
295          $stmt = $db->prepare('SELECT poster FROM events WHERE id = ? FOR UPDATE');
296          $stmt->bind_param('i', $id);
297          $stmt->execute();
298          $row = $stmt->get_result()->fetch_assoc();
299          if ($row === null) {
300              $db->rollback();
301              return ['ok' => false, 'error' => 'That event was not found.'];
302          }
303          $stmt = $db->prepare('SELECT COUNT(*) FROM registrations WHERE event_id = ? LOCK IN SHARE
     MODE');
304          $stmt->bind_param('i', $id);
305          $stmt->execute();
306          $registrations = (int) $stmt->get_result()->fetch_row()[0];
307          if ($registrations > 0) {
308              $db->rollback();
309              return ['ok' => false, 'error' => "This event has $registrations registration(s) and
     cannot be deleted, so that the registration history is kept. Close the event instead."];
310          }
311          $stmt = $db->prepare('DELETE FROM events WHERE id = ?');
312          $stmt->bind_param('i', $id);
313          $stmt->execute();
314          $db->commit();
315          return ['ok' => true, 'poster' => $row['poster']];
```

6.6 Event registration for students

On the Campus Events page a student with a linked student record sees every upcoming event with its poster, the seats (*n of capacity seats taken* or *no seat limit*) and a Register button – or *You are registered* with *Cancel registration*, *Event full* or *Registration closed*. Each button is a POST form with the CSRF token. `register_student_for_event()` runs in one transaction: it locks the event row (`SELECT … FOR UPDATE`), refuses closed and past events, inactive students and duplicates, counts the seats that are not cancelled only after the lock is held, and then inserts the registration or reactivates a cancelled one. Concurrent requests for the same event therefore wait for each other, and the capacity is never exceeded. Cancelling sets the status to `cancelled`: the row is kept as history and the seat is freed. The registrations of an event are listed for the administrator in the console (enrollment number, student, course, registration status, time).

studenthub/includes/event_registration.php (lines 45–104) – registration transaction without overbooking

```
 45  function register_student_for_event(mysqli $db, int $student_id, int $event_id): int
 46  {
 47      $db->begin_transaction();
 48      try {
 49          $stmt = $db->prepare('SELECT capacity, status, event_date < CURDATE() AS is_past FROM events
     WHERE id = ? FOR UPDATE');
 50          $stmt->bind_param('i', $event_id);
 51          $stmt->execute();
 52          $event = $stmt->get_result()->fetch_assoc();
 53          if ($event === null) {
 54              throw new EventRegistrationException(EventRegistrationException::EVENT_NOT_FOUND);
 55          }
 56          if ($event['status'] !== 'open') {
```

```
57              throw new EventRegistrationException(EventRegistrationException::EVENT_CLOSED);
58          }
59          if ((int) $event['is_past'] === 1) {
60              throw new EventRegistrationException(EventRegistrationException::EVENT_PAST);
61          }
62
63          $stmt = $db->prepare('SELECT status FROM students WHERE id = ? LOCK IN SHARE MODE');
64          $stmt->bind_param('i', $student_id);
65          $stmt->execute();
66          $student = $stmt->get_result()->fetch_assoc();
67          if ($student === null) {
68              throw new EventRegistrationException(EventRegistrationException::STUDENT_NOT_FOUND);
69          }
70          if (strcasecmp(trim((string) $student['status']), 'Inactive') === 0) {
71              throw new EventRegistrationException(EventRegistrationException::STUDENT_INACTIVE);
72          }
73
74          $stmt = $db->prepare('SELECT registration_id, status FROM registrations WHERE student_id = ?
    AND event_id = ? FOR UPDATE');
75          $stmt->bind_param('ii', $student_id, $event_id);
76          $stmt->execute();
77          $existing = $stmt->get_result()->fetch_assoc();
78          if ($existing !== null && $existing['status'] !== 'cancelled') {
79              throw new EventRegistrationException(EventRegistrationException::DUPLICATE);
80          }
81
82          if ($event['capacity'] !== null) {
83              // locking read: always counts the latest committed rows
84              $stmt = $db->prepare("SELECT COUNT(*) FROM registrations WHERE event_id = ? AND status <>
    'cancelled' LOCK IN SHARE MODE");
85              $stmt->bind_param('i', $event_id);
86              $stmt->execute();
87              if ((int) $stmt->get_result()->fetch_row()[0] >= (int) $event['capacity']) {
88                  throw new EventRegistrationException(EventRegistrationException::EVENT_FULL);
89              }
90          }
91
92          if ($existing !== null) {                                   // a cancelled registration is
    reactivated
93              $registration_id = (int) $existing['registration_id'];
94              $stmt = $db->prepare("UPDATE registrations SET status = 'registered', registered_at =
    NOW() WHERE registration_id = ?");
95              $stmt->bind_param('i', $registration_id);
96              $stmt->execute();
97          } else {
98              $stmt = $db->prepare('INSERT INTO registrations (student_id, event_id) VALUES (?, ?)');
99              $stmt->bind_param('ii', $student_id, $event_id);
100             $stmt->execute();
101             $registration_id = (int) $stmt->insert_id;
102         }
103         $db->commit();
104         return $registration_id;
```

7. Validation and Security

- **Uploads:** ten server-side checks (section 6.3), random names, a folder that is not reachable over HTTP and not executable on Apache, delivery only through `poster.php` with `Content-Security-Policy: default-src 'none'; sandbox` and `X-Content-Type-Options: nosniff`; files are replaced or deleted only after the database change succeeded.
- **Event data:** every field validated on the server (`validate_event()`), HTTP 422 with field messages; the capacity cannot drop below the seats taken.
- **SQL injection:** every query with values is a prepared statement (checked by a static test); filter values only from fixed lists; IDs matched against strict patterns.
- **XSS:** every value is printed through `e()` (`htmlspecialchars` with `ENT_QUOTES`).
- **CSRF:** every POST form (add, edit, delete, register, cancel) carries the session token; GET requests never change data.

- **Authorisation:** `events.manage` for the tab and every event action, `portal.view` for posters and the Campus Events page; only a linked student account can register.
- **Integrity:** transactions with row locks for registrations, capacity edits and deletions; UNIQUE (student_id, event_id); UNIQUE poster; category foreign key; events with registrations are never deleted.
- **Audit log:** every create, update, delete, rejected upload, registration and cancellation.

8. Output Screenshots

All screenshots are genuine full-page captures from Microsoft Edge (headless, driven by Playwright, window 1280 px wide) taken by `tests/browser_events.py` while the portal ran on PHP's built-in server with a disposable database built from `database/studenthub_demo.sql` and a temporary poster folder. The poster images are fictional sample posters from `Practical-12/uploads`. To show this case, the test gave the first demo event the older poster reference `uploads/odoo_hackathon.jpg` whose file does not exist (as for four events of the live database). Captures 02, 06, 07 and 09 of the folder are not reproduced: they show the filled add form before submission, the list after the edit, the student's event list before registering and the *Event full* state, which is also visible in Figure 6.

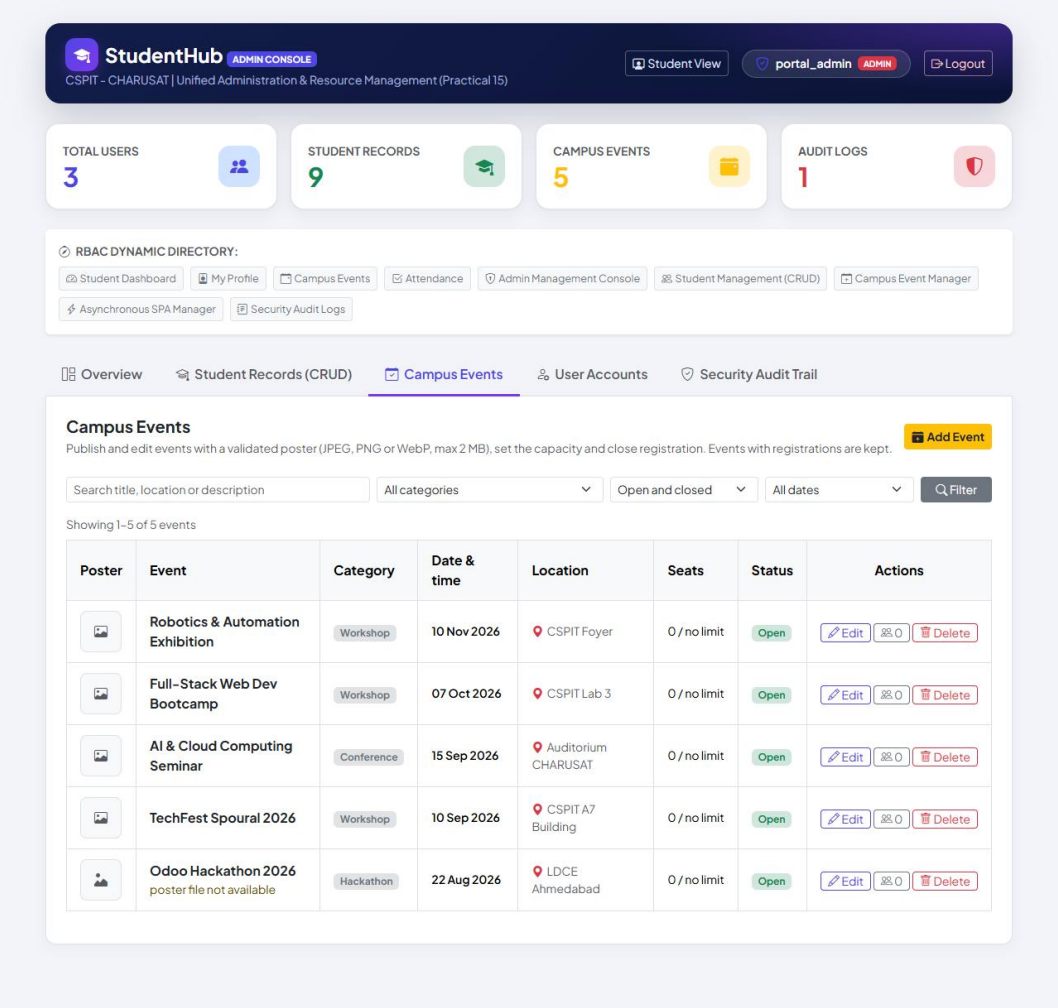


Figure 1: Campus Events tab of the Admin Console: the 5 demo events with category, date, location, seats (0 / no limit) and Open status; "Odoo Hackathon 2026" shows a generic image icon and "poster file not available" for its older, missing poster reference (01-event-list.png)

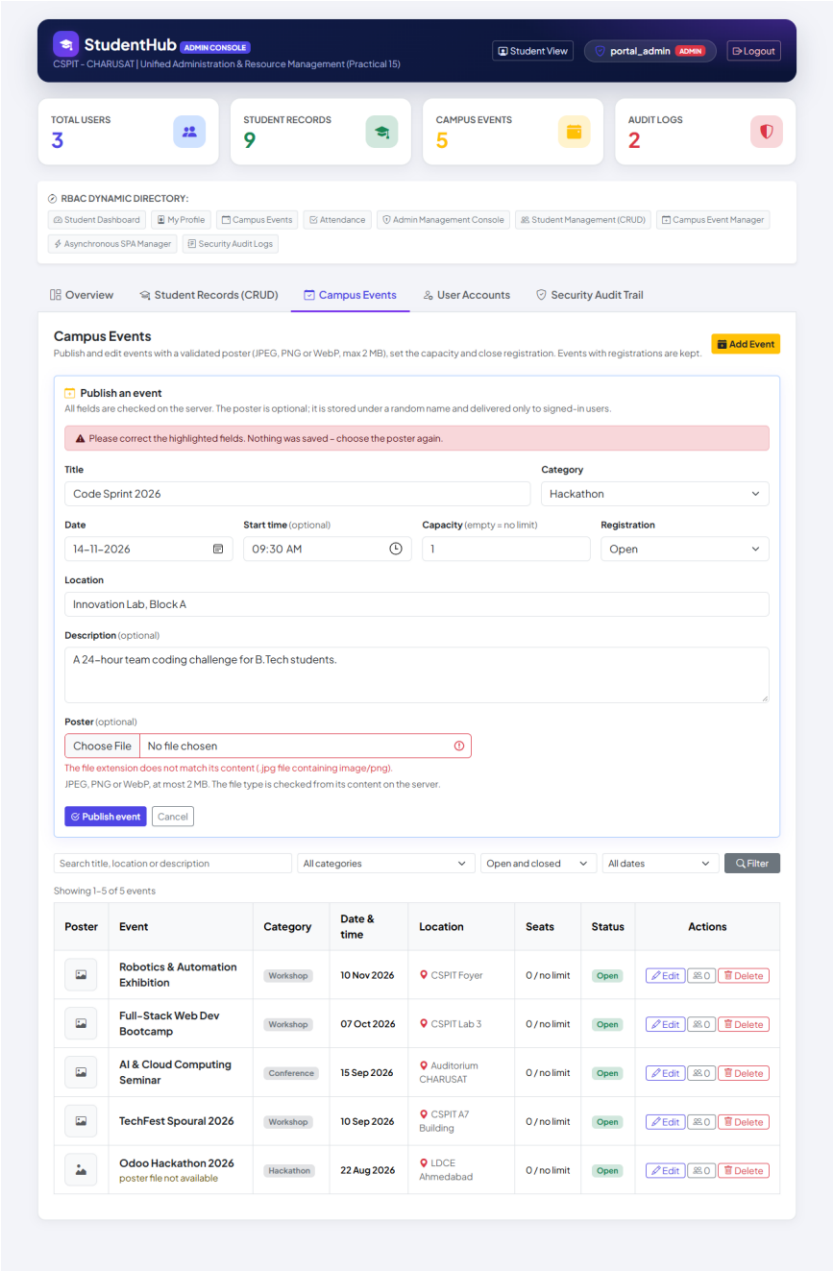


Figure 2: Publishing "Code Sprint 2026" with the file cricket-cup.jpg (a PNG image saved with a .jpg name): the server refuses it – "The file extension does not match its content (.jpg file containing image/png)." – nothing was saved and the other values are kept (03-poster-rejected.png)

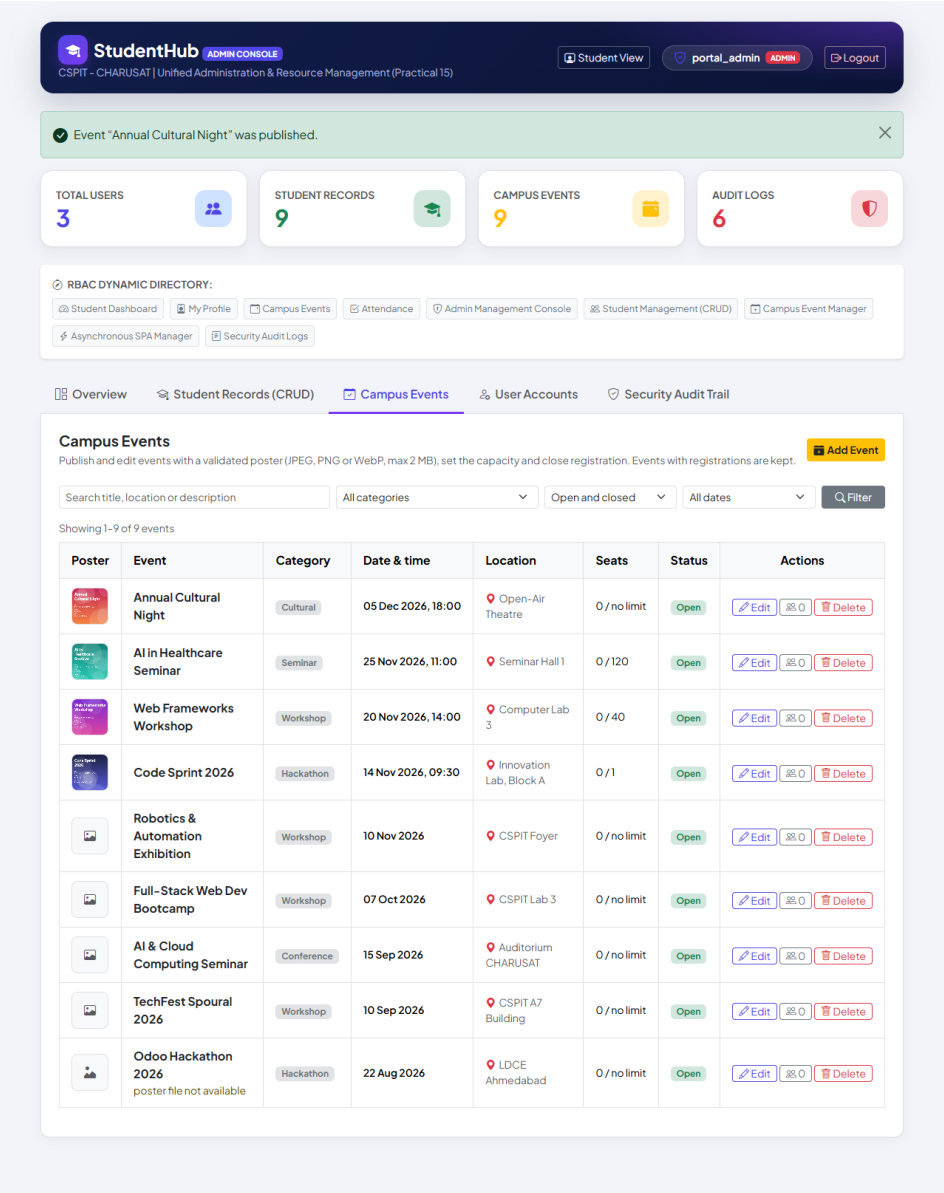


Figure 3: After four valid uploads (JPEG, PNG, WebP and JPEG posters): "Event "Annual Cultural Night" was published." – the new events show their poster thumbnails delivered by poster.php and the seats 0 / no limit, 0 / 120, 0 / 40 and 0 / 1 (04-event-created.png)

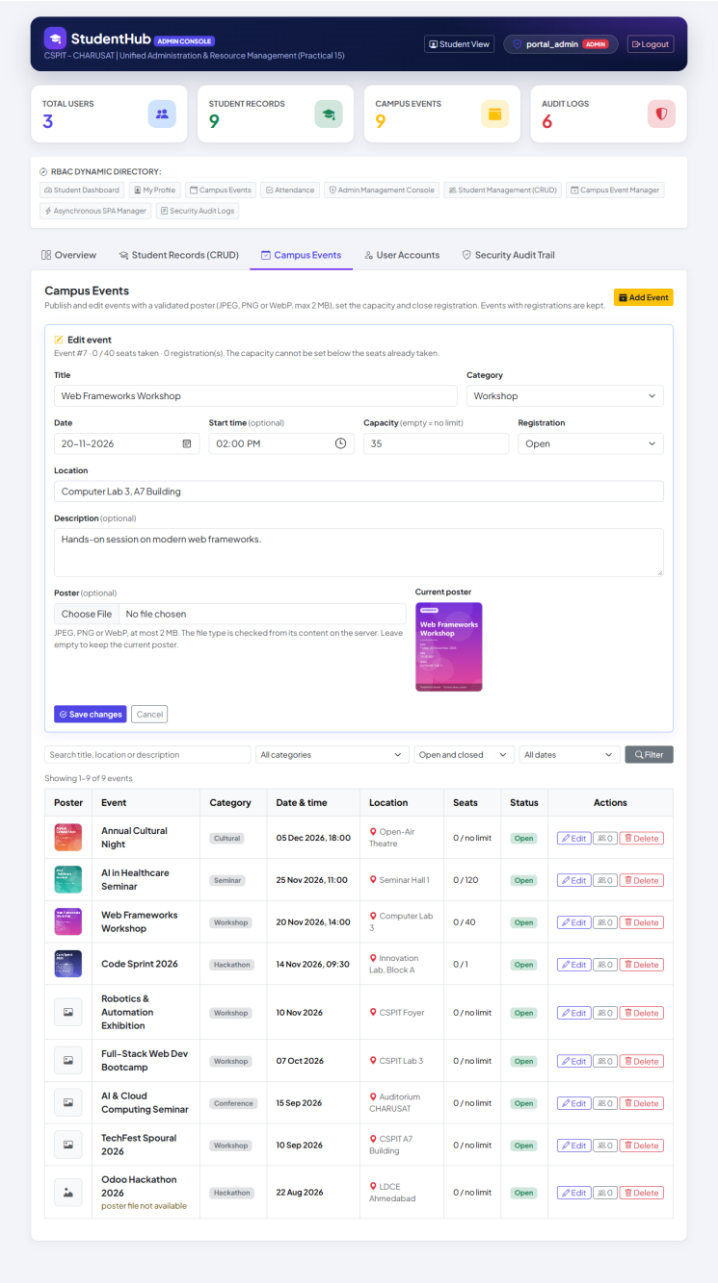


Figure 4: Editing "Web Frameworks Workshop" (event #7, 0 / 40 seats taken): capacity changed to 35, location to "Computer Lab 3, A7 Building"; the current poster is shown and kept when no new file is chosen (05-edit-event.png)

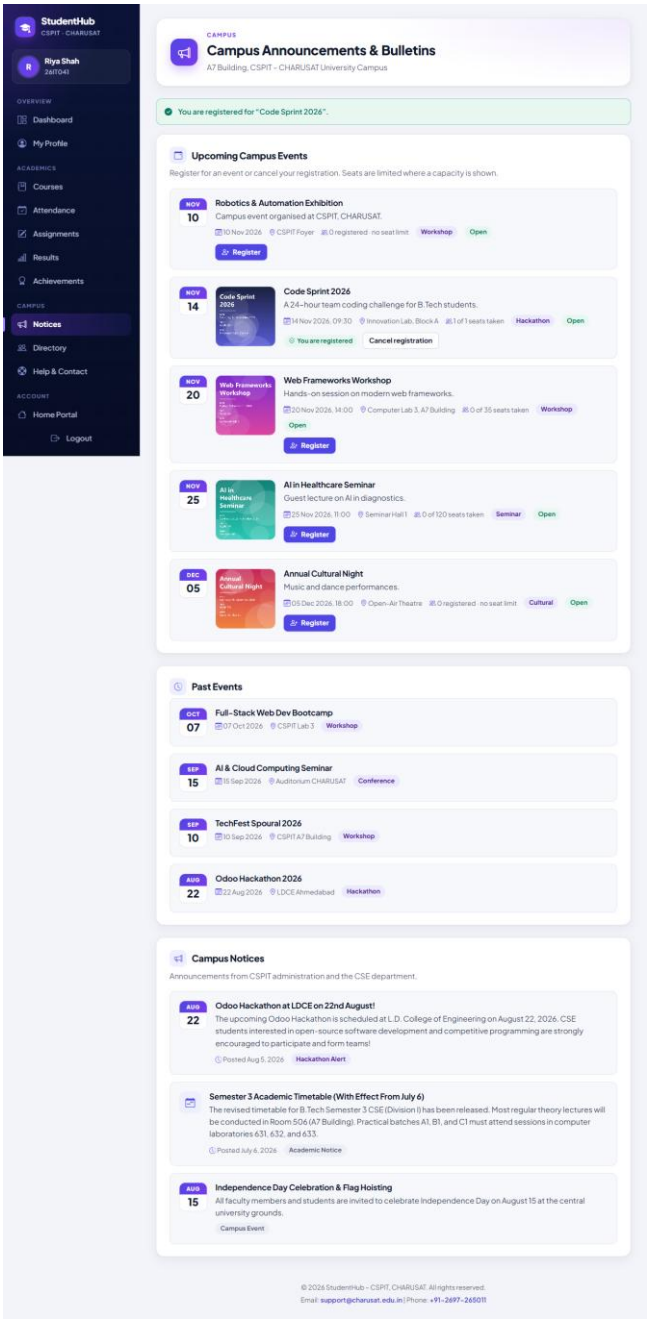


Figure 5: Campus Events page of the student Riya Shah (26IT041) after registering: success message for "Code Sprint 2026" and "You are registered" with a Cancel registration button (capacity 1) (08-registration-success.png)

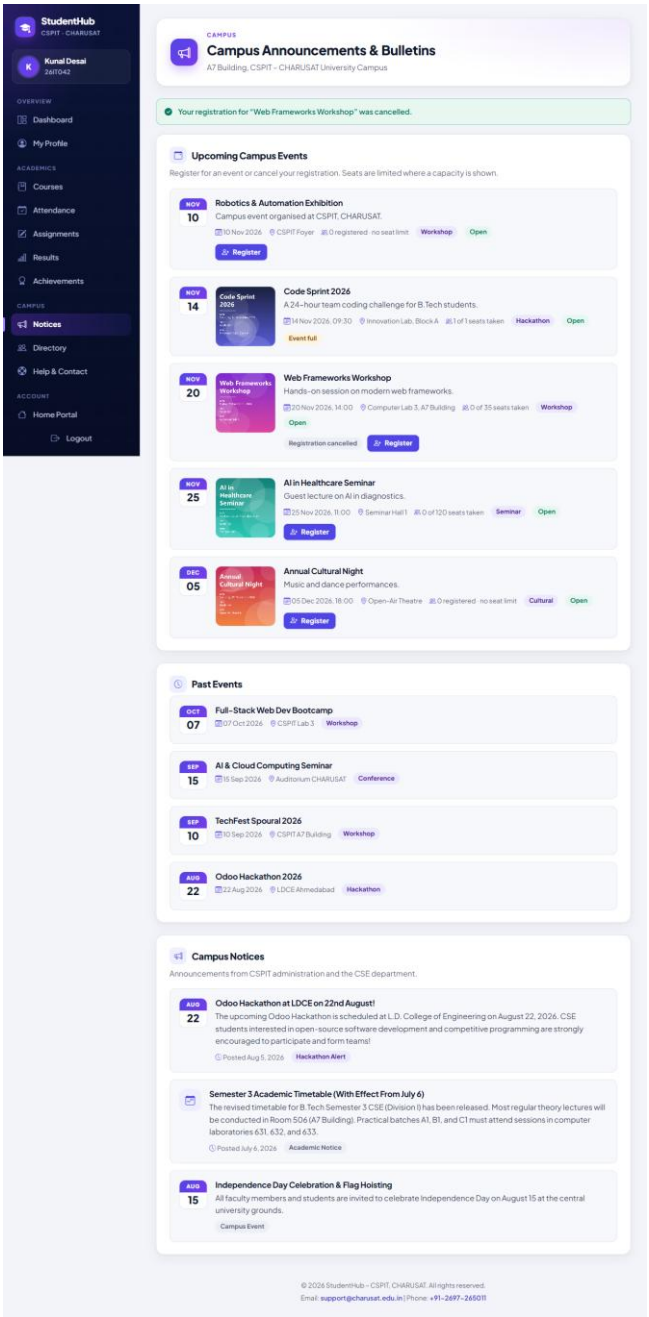


Figure 6: The same page for the second student Kunal Desai (26IT042): "Code Sprint 2026" shows 1 of 1 seats taken and "Event full" without a Register button; after registering for and cancelling "Web Frameworks Workshop" the message "Your registration … was cancelled." appears and the card shows "Registration cancelled" with Register available again (10-registration-cancelled.png)

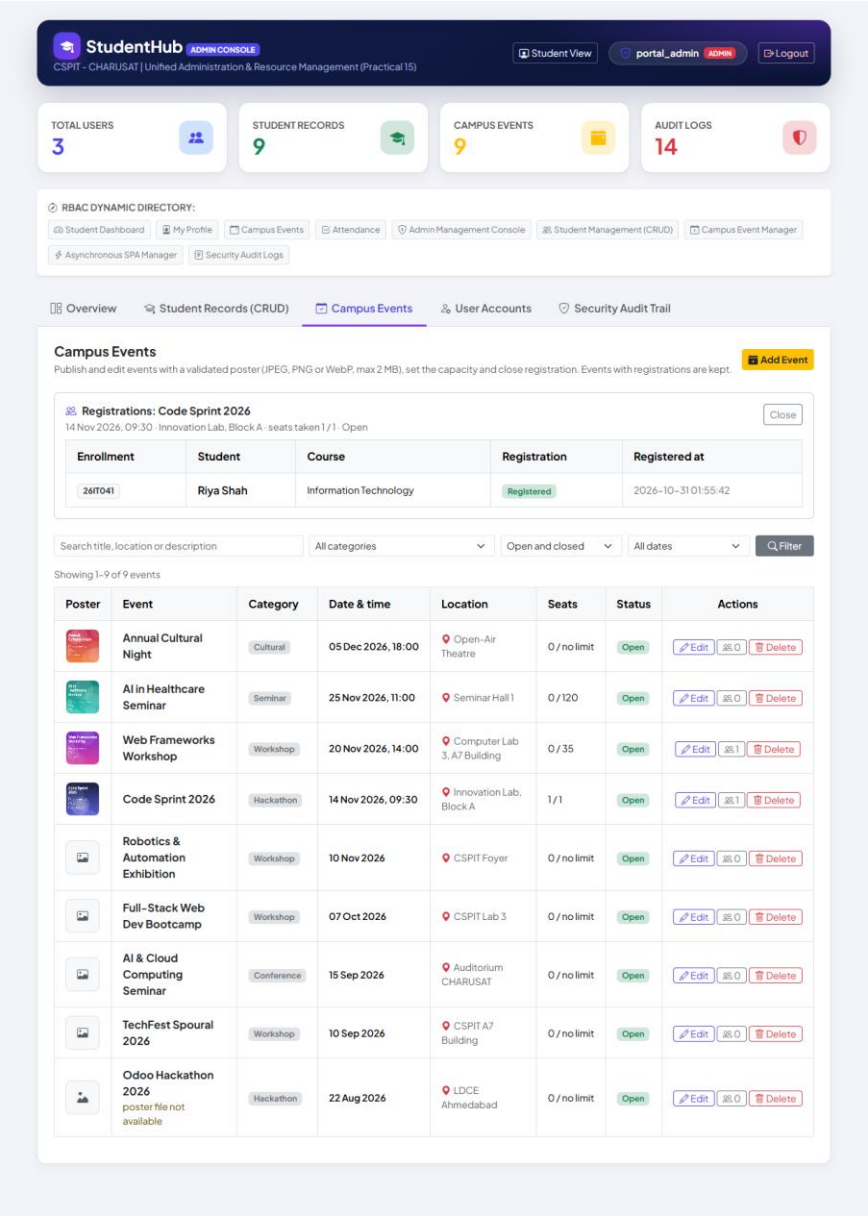


Figure 7: Registrations view in the console for "Code Sprint 2026" (seats taken 1 / 1): Riya Shah, 26IT041, Information Technology, Registered, with the registration time; the list shows 1 registration for "Web Frameworks Workshop" (the cancelled one, kept as history) and 0 / 35 seats taken (11-event-registrations.png)

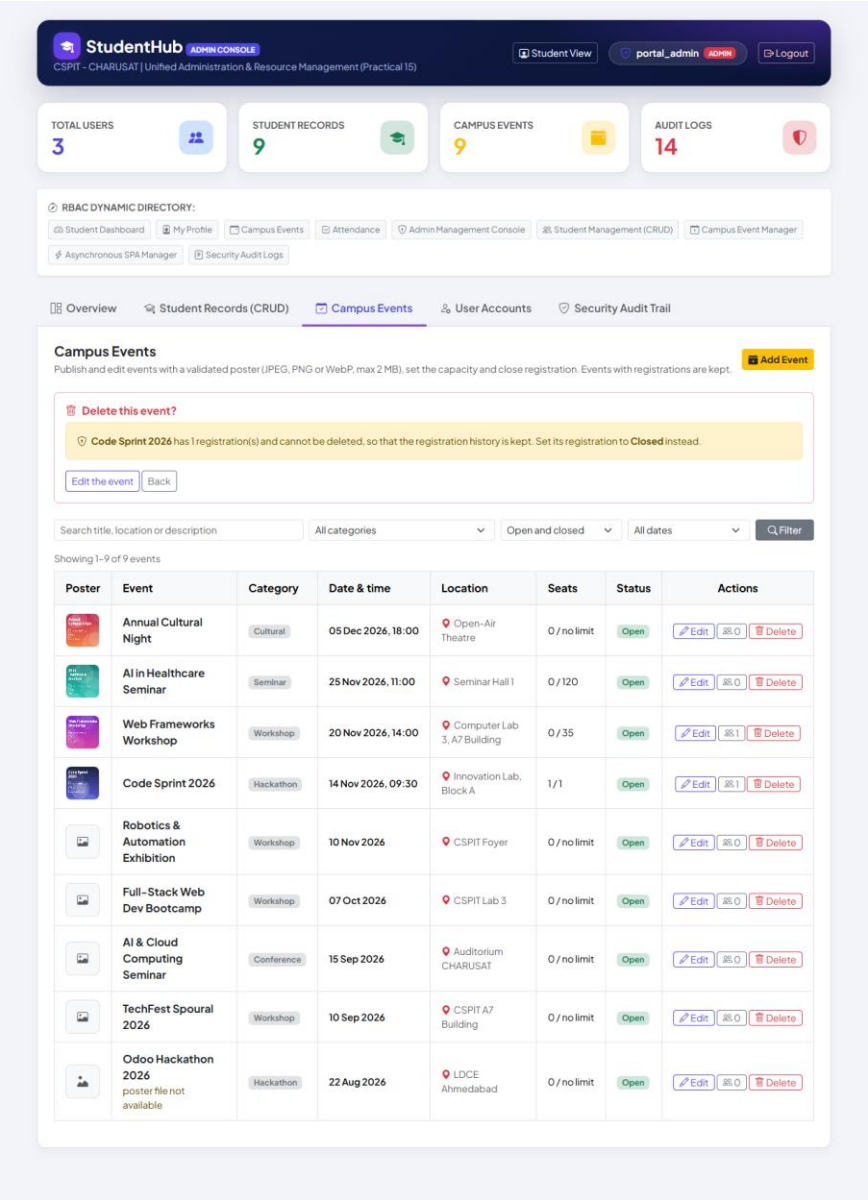


Figure 8: Delete refused: "Code Sprint 2026 has 1 registration(s) and cannot be deleted, so that the registration history is kept. Set its registration to Closed instead." – only "Edit the event" and "Back" are offered (12-delete-refused.png)

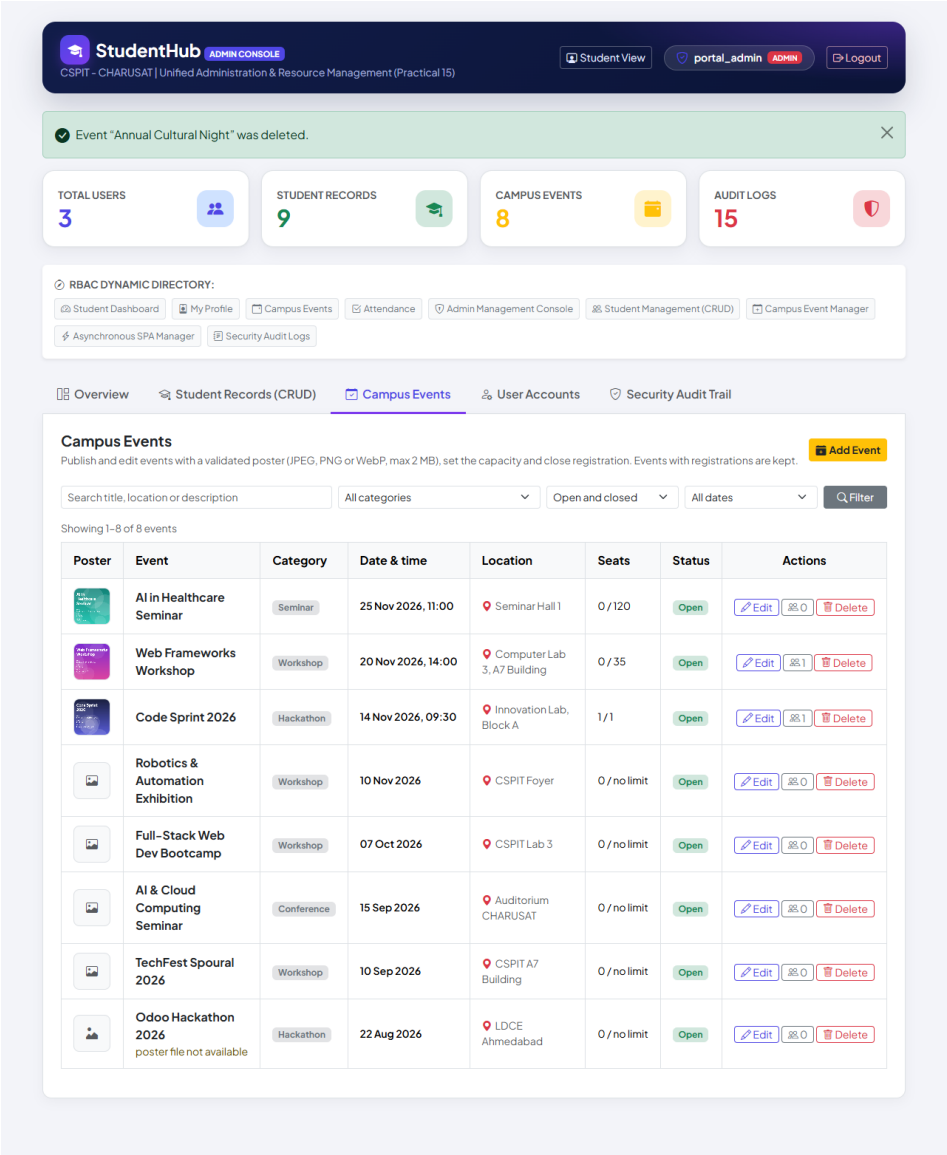


Figure 9: An event without registrations deleted after confirmation: "Event "Annual Cultural Night" was deleted." – 8 events remain; its poster file was removed from the poster folder (checked by the test) (13-event-deleted.png)

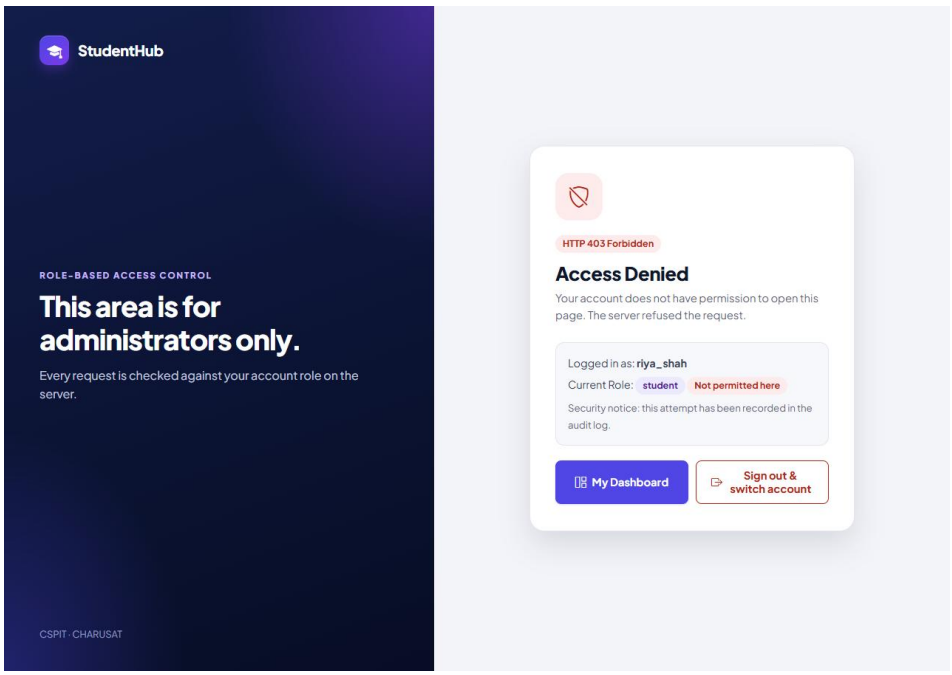


Figure 10: The student Riya Shah opening the Campus Events tab of the Admin Console: HTTP 403 "Access Denied" (14-student-access-denied.png)

9. Testing and Results

All tests were executed on 31 October 2026 on Windows 11 with PHP 8.2.12 (XAMPP), MariaDB 10.4.32 and PHP's built-in server. Write tests used disposable databases and a temporary poster folder (`STUDENTHUB_UPLOAD_DIR`); `studenthub/uploads/` and the live database were not used. The live database then received migration 003 after a verified backup; its users, password hashes, students, registrations, menu and every original event value were verified unchanged. The numbers are copied from the saved outputs and from TEST_REPORT.md §9.

Suite	Result	Output
`tests/test_events.py` – Campus Events tab, poster uploads, Campus Events page	**94 / 94 passed**	`tests/test_events_output.txt`
`tests/browser_events.py` – Microsoft Edge + screenshots	**16 / 16 passed**	`tests/browser_events_output.txt`
`database/tests/test_003_event_management.py` – fixture / real dump / `--verify-live`	**27 / 27**, **26 / 26**, **5 / 5 passed**	`database/tests/test_003_output.txt`, `test_003_verify_live_output.txt`
Regression: `test_001` (88 / 88; 24 / 24 live), `test_002` (22 / 22; 4 / 4 live), `test_auth` (132 / 132), `test_students` (170 / 170)	all passed	outputs in `tests/` and `database/tests/`
Regression in Edge: `browser_auth` (14 / 14), `browser_students` (16 / 16), `browser_inactive_login` (13 / 13)	all passed	outputs in `tests/`

9.1 Checks per area (tests/test_events.py)

Area	Checks	Examples of what is verified
Static checks	4	`php -l`; no `->query()` with values; original file name never used; `uploads/.htaccess` content
Database (migration 003)	2	`003` recorded; status, updated_at, event_categories and the foreign key exist

Event listing (admin)	5	5 events, columns, generic icon for a missing older poster, `poster.php` 404 for it, CSRF tokens
Add event	9	with and without poster; JPEG, PNG and WebP stored under 32-hex names with identical bytes; nosniff + sandbox CSP; visitors redirected; `.PNG` accepted
Invalid event details	16	empty / short title, unknown category, impossible date 2027-02-30, time 25:00, capacity 0 / −5 / 70000 / text, description over 2000
Poster upload validation	14	GIF, SVG with script, PHP, PHP renamed `.jpg`, `shell.php.jpg`, PNG named `.jpg`, truncated PNG, JPEG with PHP, 2.3 MB file, empty file, forged type; UPLOAD_REJECTED
Edit event	5	poster kept; replaced with the old file deleted; invalid replacement refused; UPDATE_EVENT
Student registration	14	success, duplicate, capacity 2 → third refused, closed, past, cancel and re-register, CSRF, GET, unlinked account, admin
Simultaneous registrations	1	6 students for 3 seats at once → exactly 3 succeed
Capacity, registrations view, deletion	7	capacity below seats taken refused; delete refused with registrations; confirmed delete removes event and poster
Search and filters	10	search, category, status, upcoming / past, combinations; SQL-injection strings as text
Access control	3	students and editors 403 for tab, forms and POST; visitors to the login page
Storage, server log, clean-up	4	only used posters remain in the folder; no PHP warnings; live database and `studenthub/uploads` unchanged
Total	**94**	

The Edge test (16 checks) published events with real JPEG, PNG and WebP posters, saw the server refuse a PNG renamed to `.jpg`, edited an event, registered two students until the capacity of 1 was reached, cancelled a registration, opened the registrations view, saw a delete refused for an event with registrations and an event without registrations deleted together with its poster, confirmed HTTP 403 for a student and found no JavaScript errors and no PHP warnings (apart from the known, pre-existing 404 for `/favicon.ico`).

10. Problems Found and Handled During Integration

- **Windows security software:** for an uploaded test file that contains a well-known web-shell signature, the security software blocks reading the file. The portal then rejects it with "could not be read as an image" instead of the content-type message – the file is rejected either way (TEST_REPORT §9). `validate_poster()` reads the file once and rejects an unreadable file cleanly.
- **Older poster references:** existing events store paths such as `uploads/odoo_hackathon.jpg` whose files do not exist. Instead of broken images or file access outside the poster folder, `poster_path()` accepts only stored random names; these events show a generic image icon, `poster.php` answers 404, and the value is kept until a new poster is uploaded.
- **Registration history and CASCADE:** because `registrations` references `events` with `ON DELETE CASCADE`, deleting an event would erase its registrations. Deletion is therefore refused for events with registrations (closing the event is offered instead), both in the panel and inside `delete_event()`.
- **Concurrent registrations:** a plain INSERT would bypass the capacity rule; all registrations go through `register_student_for_event()`, verified with 6 simultaneous requests for 3 seats (exactly 3 succeed).

11. Differences from the Original Standalone Practical and Limitations

Aspect	Original report (Practical-12/)	Integrated portal (studenthub/)

Database	Own `studenthub_prac12` with `events`, `event_categories`, `admins`	Portal `studenthub_db.events` extended by migration 003; categories table with FK
Login	Own administrator table and session cookie `P12SESSID`	Portal login (`STUDENTHUBSESSID`); role admin with `events.manage`
Event fields	event_title, required description (10–2000), venue, required time and poster	title, location, optional time, description and poster; capacity 1–65535 or no limit
Poster column	`poster_filename` VARCHAR(64), UNIQUE + CHECK REGEXP	`poster` VARCHAR(255), UNIQUE; name pattern enforced by `poster_path()` (older values kept)
poster.php	Validated file name, administrators only	Event ID, every signed-in user (`portal.view`)
Delete	Event and poster after confirmation	Only events without registrations; otherwise close the event
Registration	–	Students register and cancel on the Campus Events page (transaction, capacity, duplicates)
Orphan clean-up	`tools\cleanup_orphan_posters.php`	No separate tool; new files are removed again when saving fails and old files after a replacement
Tests	153 / 153 automated, 13 / 13 browser	94 / 94 automated, 16 / 16 browser, migration 003 tests

Limitations (TEST_REPORT §9): registrations cannot be marked *attended* from the interface; there is no waiting list; the Campus Events page shows all upcoming events on one page. The `.htaccess` protection of the poster folder takes effect only on Apache (`DEPLOYMENT.md` recommends placing the folder outside the document root via `STUDENTHUB_UPLOAD_DIR`); the portal was tested with PHP's built-in server, which ignores `.htaccess`, so the tests checked the file's content. The Admin Console loads Bootstrap from a CDN.

12. Conclusion

Event management now works on the portal's own events. Administrators publish, edit, filter and delete events in the Admin Console; posters are accepted only after ten server-side checks of the upload and its content, stored under random names in a folder that is never served or executed, delivered only by `poster.php`, and replaced or removed only after the database change succeeded. Students register for events and cancel on the Campus Events page; one transaction with row locks prevents duplicates and overbooking, and events with registrations are never deleted, so the registration history is kept. Prepared statements, output escaping, CSRF tokens, server-side permissions and the audit log protect the module; 94 automated checks, 16 browser checks and the migration 003 tests passed.

13. References

- Project repository: https://github.com/TheEmperor08/StudentHub-ITUE203-25CS023 – `studenthub/`, `database/` (migration 003), `tests/`, `evidence/practical-12-integration/`, `TEST_REPORT.md` §9, `DEPLOYMENT.md`; commit `ba11795`
- PHP Manual – Handling file uploads: https://www.php.net/manual/en/features.file-upload.php
- PHP Manual – Fileinfo (finfo): https://www.php.net/manual/en/book.fileinfo.php
- PHP Manual – getimagesizefromstring: https://www.php.net/manual/en/function.getimagesizefromstring.php
- PHP Manual – random_bytes: https://www.php.net/manual/en/function.random-bytes.php
- PHP Manual – MySQLi prepared statements: https://www.php.net/manual/en/mysqli.quickstart.prepared-statements.php
- MariaDB Knowledge Base – SELECT … FOR UPDATE: https://mariadb.com/kb/en/for-update/

- OWASP – File Upload Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html
- OWASP – Cross-Site Request Forgery Prevention Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html